PRZEWODNIK DO SAMODZIELNEJ NAUKI

Databricks Free Edition

Szkolenie krok po kroku we własnym, bezpłatnym środowisku

Altkom Akademia
Wydanie: październik 2026

Spis treści

CZĘŚĆ 1	Zanim zaczniesz	3
CZĘŚĆ 2	Konfiguracja workspace'u	4
CZĘŚĆ 3	Praca z notebookami	6
CZĘŚĆ 4	Szkolenie krok po kroku	7
CZĘŚĆ 5	Joby we Free Edition	10
CZĘŚĆ 6	Różnice względem szkolenia z trenerem	12
CZĘŚĆ 7	Rozwiązywanie problemów	13
CZĘŚĆ 8	Sprzątanie	14

Ten przewodnik pokazuje, jak samodzielnie przejść szkolenie Databricks Fundamental w Databricks Free Edition. Free Edition jest bezpłatna i nie wymaga subskrypcji chmurowej. Moduły, oba warsztaty i projekt końcowy są przygotowane tak, aby działały w niej tak samo jak na klasycznym klastrze używanym na zajęciach. Status testów: w toku. Pełnej ścieżki Free Edition dla tej edycji szkolenia nie uruchomiono jeszcze od początku do końca; jeśli komórka zakończy się błędem, zajrzyj najpierw do części 7.

Pozycja	Szczegóły
Czego potrzebujesz	Konta Databricks Free Edition i przeglądarki
Czas konfiguracji	Około 10 minut
Czas szkolenia	395 minut zajęć i warsztatów w dniu z trenerem; samodzielnie zwykle dłużej, a sekcje BONUS dokładają kolejny czas
Compute	Tylko Serverless – nie tworzysz żadnego klastra
Repozytorium szkolenia	`https://github.com/ditw-training/databricks-fundamental`

CZĘŚĆ 1

Zanim zaczniesz

Czym jest Free Edition

Databricks Free Edition to bezpłatny workspace do nauki, przeznaczony do użytku niekomercyjnego. Zawiera tę samą platformę, z której korzysta się w pracy: notebooki, Unity Catalog, Delta Lake, Lakeflow Jobs, SQL Editor i AI/BI. Logujesz się jednorazowym kodem wysłanym na e-mail albo kontem Google lub Microsoft.

Ograniczenia ważne dla tego szkolenia

Ograniczenie	Co to oznacza dla ciebie
Wyłącznie compute Serverless	Notebooki i joby zawsze działają na Serverless. Kilka demonstracji BONUS działa inaczej (zob. część 6)
Jeden SQL warehouse o rozmiarze 2X-Small	Wystarcza na SQL Editor, dashboardy i Genie Agents w tym szkoleniu
Najwyżej 5 tasków jobów działających jednocześnie	Joby ze szkolenia mają po trzy taski – uruchamiaj jeden job naraz
Jeden workspace, brak konsoli konta	Nie utworzysz grup kontowych; demo Unity Catalog używa wbudowanej grupy
Brak własnych lokalizacji storage	Twój katalog jest przechowywany we wbudowanym storage workspace'u
Dzienny limit zużycia	Po jego przekroczeniu compute jest wyłączany do następnego dnia. Dane, notebooki i ustawienia zostają
Tylko użytek niekomercyjny	Workspace służy do nauki, nie do danych firmowych

Databricks może zmieniać te limity; aktualna lista jest w dokumentacji Databricks w rozdziale *Free Edition limitations*.

Jak zbudowane jest szkolenie

Repozytorium zawiera trzy rodzaje notebooków:

Folder	Zawartość	Jak z niego korzystać
`notebooks/setup`	Konfiguracja środowiska	Uruchom raz `00_pre_config`, potem raz `00_setup`
`notebooks/modules`	Moduły demonstracyjne M00–M07	Czytaj objaśnienia i uruchamiaj komórki po kolei
`notebooks/workshops`	Ćwiczenia praktyczne i projekt końcowy	Pisz własny kod, po każdym zadaniu uruchom komórkę CHECK

Dane pochodzą z fikcyjnego sklepu internetowego RetailHub: klienci, zamówienia i produkty w plikach CSV, JSON, Parquet i Excel w folderze `dataset`.

CZĘŚĆ 2

Konfiguracja workspace'u

Krok 1 – Załóż konto Free Edition

1. Otwórz `https://www.databricks.com/learn/free-edition` i wybierz **Sign up**.
2. Zaloguj się adresem e-mail, kontem Google albo Microsoft.
3. Poczekaj, aż otworzy się workspace. Zawiera już katalogi `workspace`, `samples` i `system`.

Nazwa twojego katalogu szkoleniowego pochodzi od konta, którym się logujesz, więc przez całe szkolenie korzystaj z tego samego konta.

Krok 2 – Sklonuj repozytorium jako Git folder

1. W menu po lewej otwórz **Workspace**, a następnie **Home** (twój folder użytkownika).
2. Wybierz **Create**, a potem **Git folder**.
3. Wpisz adres repozytorium `https://github.com/ditw-training/databricks-fundamental`. Dostawca GitHub zostanie rozpoznany automatycznie. Repozytorium jest publiczne, więc nie są potrzebne żadne dane logowania.
4. Wybierz **Create Git folder**.

W folderze Home pojawi się folder `databricks-fundamental` z podfolderami `notebooks`, `dataset`, `materials`, `assets` i `docs`.

> Trzymaj notebooki w Git folderze. Notebooki odnajdują notebook konfiguracyjny i pliki z danymi po ścieżkach względnych w repozytorium; notebook skopiowany gdzie indziej zatrzyma się na `%run ../setup/00_setup`.

Krok 3 – Uruchom `00_pre_config`

1. Otwórz `notebooks/setup/00_pre_config`.
2. Notebook jest napisany dla trenera, który przygotowuje katalogi dla całej grupy. We Free Edition uruchamiamy go dla jednej osoby: w komórce konfiguracyjnej (Step 0) ustaw `SINGLE_USER_MODE = True`. Grupę szkoleniową i lokalizację storage zostaw bez zmian; tryb jednoosobowy z nich nie korzysta.
3. W prawym górnym rogu wybierz **Connect**, a potem **Serverless**.
4. Wybierz **Run all**.

Notebook wypisuje, w jakim trybie działa i co dostosował, na przykład że katalog korzysta z wbudowanego storage i której grupy użyje moduł Unity Catalog. Ostatnia komórka informuje, czy środowisko jest gotowe. Notebook utworzył:

Obiekt	Nazwa
Katalog	`retailhub_<nazwa>`
Schematy	`bronze`, `silver`, `gold`, `default`
Volume	`retailhub_<nazwa>.default.datasets` z kopią folderu `dataset`
Uprawnienia	`ALL PRIVILEGES` i `MANAGE` na katalogu dla twojego konta

Ponowne uruchomienie `00_pre_config` jest bezpieczne: tworzy tylko to, czego brakuje, i jeszcze raz kopiuje pliki z danymi.

Nazwa twojego katalogu

Katalog nazywa się `retailhub_<nazwa>`, gdzie `<nazwa>` pochodzi z twojego adresu e-mail. Nie trzeba jej ustalać ręcznie: `00_setup` wypisuje ją za ciebie (krok 4 i krok 5). Wypisanej wartości używaj wszędzie, gdzie przewodnik albo notebooki podają `retailhub_<nazwa>`, na przykład w parametrach jobów i w SQL Editorze.

Krok 4 – Uruchom `00_setup`

Otwórz `notebooks/setup/00_setup`, połącz się z Serverless i wybierz **Run all**. Notebook sprawdza katalog, schematy i volume, a na końcu wypisuje podsumowanie, na przykład:

```
TRAINING ENVIRONMENT READY
  CATALOG:        retailhub_<nazwa>
  DATASET_PATH:   /Volumes/retailhub_<nazwa>/default/datasets
  IS_SERVERLESS:  True
```

Każdy moduł i warsztat uruchamia ten notebook poleceniem `%run ../setup/00_setup` i dostaje zmienne `CATALOG`, `BRONZE_SCHEMA`, `SILVER_SCHEMA`, `GOLD_SCHEMA`, `DATASET_PATH` i `IS_SERVERLESS`. Nie musisz go już uruchamiać ręcznie.

Krok 5 – Sprawdź środowisko

Otwórz `notebooks/modules/00_intro` i uruchom dwie komórki w sekcji *Environment Check*. Pierwsza wykonuje `%run ../setup/00_setup`, druga wypisuje nazwę twojego katalogu i ścieżkę do danych. Zapisz nazwę katalogu; będzie potrzebna w części 5.

CZĘŚĆ 3

Praca z notebookami

Compute

Zawsze podłączaj notebooki do **Serverless**. Uruchamia się w kilka sekund i sam się zatrzymuje, gdy nie pracujesz, więc nie trzeba niczego wyłączać. Dla SQL Editora, dashboardów i Genie Agents używaj **Serverless Starter Warehouse**.

Uruchamianie komórek

Uruchamiaj komórki modułów pojedynczo i czytaj objaśnienie nad każdą z nich — tak jest napisane szkolenie. **Run all** też działa, ale mniej się z niego nauczysz. Moduł można uruchomić ponownie w dowolnym momencie: komórki demonstracyjne same odtwarzają swoje tabele.

Oznaczenia w notebookach

Oznaczenie	Znaczenie	Co zrobić
⊕ **BONUS**	Poza jednodniowym planem	Pomiń za pierwszym razem, wróć później
⭐ **OPTIONAL**	Zadanie warsztatowe poza ścieżką CORE	Wykonaj, jeśli masz czas
[UI DEMO]	Krok wykonywany w interfejsie Databricks, nie w kodzie	Przejdź listę kroków z komórki
`%skip` na początku komórki	Komórka nie jest uruchamiana przez **Run all**	Usuń tę linię, gdy tekst każe to zrobić
`# YOUR CODE HERE`	Miejsce na twój kod w warsztacie	Zastąp je swoim rozwiązaniem
Komórka **CHECK**	Automatyczny test poprzedniego zadania	Uruchom ją; wszystkie asercje muszą przejść, zanim pójdziesz dalej

Warsztaty

Każde zadanie warsztatowe ma komórkę zadania i komórkę CHECK. Napisz kod, uruchom komórkę zadania, potem komórkę CHECK. Nieudana asercja mówi, co jest nie tak. Popraw komórkę zadania i uruchom obie komórki jeszcze raz.

Zadania oznaczone ⭐ OPTIONAL wykraczają poza ścieżkę CORE. Jeśli komórka zadania opcjonalnego albo jego komórka CHECK zaczyna się od `%skip`, usuń tę linię w obu komórkach, żeby je uruchomić.

Warsztaty zapisują do własnych tabel (`lab_ingestion_customers`, `lab_delta_cust_ws`, `lab_orders`, `lab_daily_orders`), więc nigdy nie nadpisują tabel demonstracyjnych.

Pobieranie aktualizacji

Gdy repozytorium szkolenia zostanie zaktualizowane, otwórz okno Git (wybierz **Git** obok nazwy folderu albo przycisk gałęzi `main` w notebooku) i wybierz **Pull**.

Konflikt przy Pull oznacza, że notebook zmieniony lokalnie zmienił się też w repozytorium. Skopiuj swoją wersję poza Git folder (**Clone** w menu notebooka), a potem w oknie Git wybierz ⋮, **Reset**, aby odrzucić lokalne zmiany, i ponownie **Pull**.

CZĘŚĆ 4

Szkolenie krok po kroku

Kolejność odpowiada jednodniowemu planowi szkolenia. Czasy pochodzą ze szkolenia z trenerem; samodzielnie zwykle zajmuje to więcej. W każdym module wszystko po podsumowaniu (Summary), pod banerem ⊕ BONUS, jest dodatkiem.

M00 – Wprowadzenie i konfiguracja (15 min)

`notebooks/modules/00_intro`. Cele, agenda, dane RetailHub i sprawdzenie środowiska z części 2, krok 5.

M01 – Platforma, koszty i interfejs (65 min)

`notebooks/modules/01_platform_workspace`. Workspace, Catalog Explorer, trzypoziomowa przestrzeń nazw, volume'y, rodzaje compute i ich koszty, magic commands, Genie Code oraz wykresy i profil danych bez kodu.

Do zrobienia ręcznie:

- **Upload plików do volume.** Pobierz `stores.csv` i `promotions.json` z folderu `materials/volume_upload` repozytorium na GitHubie, a potem w **Catalog** wgraj je do volume wskazanego w notebooku (**Upload to this volume**). Możesz też uruchomić komórkę kopiującą w notebooku, która skopiuje je z Twojego Git folderu. Uruchom kolejne komórki, żeby odczytać pliki.
- **Wizualizacje i profil danych.** Przy wyniku `display()` wybierz +, a potem **Visualization** albo **Data Profile**.

Uwagi dla Free Edition: zapytanie o koszty na `system.billing.usage` to demo trenera. Jeśli we Free Edition zakończy się błędem uprawnień albo braku tabeli, pomiń je. Na Serverless Photon jest zawsze włączony.

M02 – Jak działa Spark (15 min)

`notebooks/modules/02_spark_architecture`. Driver i executory, lazy evaluation oraz jeden silnik dla SQL i PySpark, na tabeli `samples.nyctaxi.trips`. BONUS obejmuje stage, shuffle, AQE, Spark UI i cache.

Uwagi dla Free Edition:

- Serverless pozwala odczytać tylko część ustawień Spark, więc komórka BONUS z AQE pokazuje dla pozostałych wartości domyślne.
- Cache DataFrame'ów na Serverless jest w Public Preview od października 2026 r. Wymaga środowiska serverless w wersji 5 lub nowszej, jest domyślnie włączony, a administrator workspace może go wyłączyć. Na Serverless `cache()` od razu oblicza wynik i trzyma go jako snapshot na czas twojej sesji; SQL `CACHE TABLE` nie jest obsługiwane. Jeśli cache nie jest dostępny, komórka BONUS wypisze komunikat zamiast błędu.
- Nie ma Spark UI. Zamiast niego użyj profilu zapytania: **See performance** pod wynikiem komórki, a potem **See query profile**.

M03 – Medallion i ingestion (55 min)

`notebooks/modules/03_medallion_ingestion`. Pięciominutowe wprowadzenie do Pythona, architektura medallion, CSV i JSON z jawnym schematem, `read_files()`, eksploracja danych, transformacje z odpowiednikami w SQL, widoki tymczasowe i zapis do tabeli Delta. Parquet i Excel są w sekcjach BONUS.

Uwagi dla Free Edition: pliki Excel odczytuje wbudowany czytnik Excel, który na klasycznym compute wymaga Databricks Runtime 17.1 lub nowszego. BONUS *Global Temporary Views* jest pomijany — Serverless nie obsługuje globalnych widoków tymczasowych. Zwykłe widoki tymczasowe działają.

Warsztat – Ingestion (40 min)

`notebooks/workshops/WORKSHOP_ingestion`. Zadania CORE prowadzą od odczytu CSV i JSON ze schematem, przez nowe kolumny, rzutowanie typów, logikę warunkową i agregacje, do zapisu tabeli Bronze w Delta z metadanymi ładowania. Pozostałe zadania są ⭐ OPTIONAL; obie grupy wymienia tabela *Tasks at a Glance* w notebooku.

M04 – Delta Lake (40 min)

`notebooks/modules/04_delta_fundamentals`. Tworzenie tabeli, schema enforcement i evolution, INSERT, UPDATE, DELETE, MERGE, historia tabeli, time travel i RESTORE. VACUUM, UNDROP, liquid clustering i predictive optimization są omówione jako pojęcia.

Uwagi dla Free Edition: demo BONUS z VACUUM skraca retencję własnej tabeli demonstracyjnej, a potem ją przywraca. Po VACUUM starych wersji nie da się już odczytać i o to chodzi w tym demo. BONUS *External Tables* informuje, że nie ma dostępnej external location — we Free Edition to oczekiwane.

Warsztat – Delta Lake (35 min)

`notebooks/workshops/WORKSHOP_delta`. Zadania CORE 01, 03, 05 i 06, 08, 10 i 12: utworzenie tabeli Delta, dodanie kolumny przez schema evolution, zmiany danych przez INSERT, UPDATE i MERGE, odczyt starszej wersji przez time travel i odtworzenie tabeli poleceniem RESTORE. Zadania 02, 04, 07, 09 i 11 (schema enforcement, ograniczenie CHECK, DELETE, historia, time travel po znaczniku czasu) są ⭐ OPTIONAL.

M05 – Lakeflow Jobs (35 min)

`notebooks/modules/05_orchestration_jobs`. Lakeflow Jobs (wcześniej Workflows): compute dla jobów, job z trzema taskami zbudowany z gotowych notebooków, parametry joba i taska, task values, harmonogram i repair run. Lakeflow Spark Declarative Pipelines i monitoring jobów w tabelach systemowych są w sekcjach BONUS. Zbuduj job opisany w części 5.

M06 – Unity Catalog (15 min)

`notebooks/modules/06_unity_catalog`. Grupy kontowe, potem `GRANT`, `SHOW GRANTS` i `REVOKE`, a na końcu to samo w zakładce **Permissions** w Catalog Explorer.

Uwagi dla Free Edition: nie ma konsoli konta, więc grupa `retailhub_analysts` nie może istnieć. Postępuj zgodnie z uwagą dla trybu jednoosobowego w notebooku: w tym trybie demo nadaje uprawnienia wbudowanej grupie `account users`, a `SHOW GRANTS` i `REVOKE` działają tak samo. We Free Edition jesteś jedynym członkiem `account users`.

Projekt końcowy (65 min)

`notebooks/workshops/final_labs/WORKSHOP_final_project`. Budujesz pipeline zamówień od surowego JSON do tabeli z dziennym przychodem i uruchamiasz go jako job z harmonogramem.



1. Uruchom `tasks/lab_task_01_bronze` — jest gotowy.
2. Uzupełnij i uruchom `tasks/lab_task_02_silver` oraz `tasks/lab_task_03_gold`; uruchom komórki CHECK.
3. Wykonaj kroki z notebooka projektu: smoke test, job opisany w części 5 i zapytania kontrolne w SQL.

Oczekiwane wyniki:

Tabela	Liczba wierszy
`bronze.lab_orders`	100 000
`silver.lab_orders`	90 904 (błędne wiersze są usuwane)
`gold.lab_daily_orders`	32 (jeden wiersz na dzień, od 2024-12-31 do 2025-01-31)

M07 – Genie Agents i dashboardy (opcjonalnie)

`notebooks/modules/07_aibi_dashboards`. Najpierw uruchom komórki notebooka: tworzą tabele demonstracyjne w twoim schemacie Gold. Potem wykonaj w interfejsie kroki opisane w notebooku: Genie Agent (wcześniej Genie space) na tabelach Gold i dashboard AI/BI. Używaj Serverless Starter Warehouse.

CZĘŚĆ 5

Joby we Free Edition

Joby tworzysz w **Jobs & Pipelines**, wybierając **Create**, a potem **Job**. We Free Edition jedynym compute dla taska jest **Serverless**. Wszędzie, gdzie notebooki wspominają *Shared training cluster* albo *Job cluster*, wybierz Serverless.

Job demonstracyjny z M05

Trzy notebooki tasków są gotowe w `materials/orchestration`.

Nazwa taska	Notebook	Zależy od
`validate`	`materials/orchestration/task_01_validate`	–
`transform`	`materials/orchestration/task_02_transform`	`validate`
`report`	`materials/orchestration/task_03_report`	`transform`

1. Utwórz job i nadaj mu nazwę, na przykład `Demo_ETL_Pipeline_<nazwa>`.
2. Dodaj trzy taski z tabeli. Dla każdego: typ **Notebook**, źródło **Workspace**, compute **Serverless**.
3. Dodaj parametry joba `source_table` = `samples.nyctaxi.trips` i `min_rows` = `100`. Parametry joba trafiają do każdego taska; jeśli parametr taska ma ten sam klucz, wygrywa parametr joba.
4. Dodaj trigger: **Scheduled**, **Advanced**, cron `0 0 6 * * ?`, strefa czasowa `Europe/Warsaw`, status **Paused**.
5. Wybierz **Run now**, otwórz przebieg i obejrzyj graf tasków oraz wynik każdego z nich.
6. Wybierz **Run now with different settings** i ustaw `min_rows` = `999999999`: `validate` zakończy się błędem. Użyj **Repair run** z `min_rows` = `100`; uruchomią się ponownie tylko task z błędem i taski po nim.

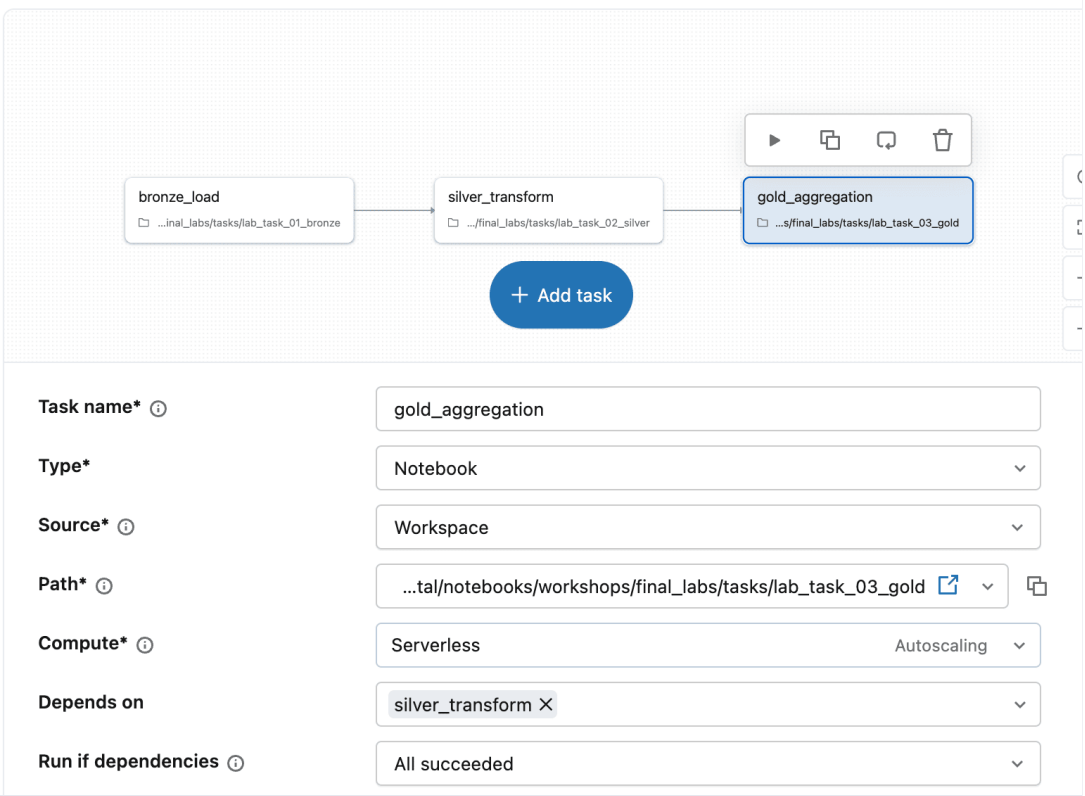
> Nazwy tasków mają znaczenie. `transform` odczytuje task value z taska o nazwie `validate`, a `report` — z `transform`. Przy innych nazwach task zależny zakończy się błędem. Zmień nazwę taska i użyj **Repair run**.

Job projektu końcowego

Nazwa taska	Notebook	Zależy od
`bronze_load`	`notebooks/workshops/final_labs/tasks/lab_task_01_bronze`	–
`silver_transform`	`notebooks/workshops/final_labs/tasks/lab_task_02_silver`	`bronze_load`
`gold_aggregation`	`notebooks/workshops/final_labs/tasks/lab_task_03_gold`	`silver_transform`

Parametry joba, z nazwą katalogu wypisaną przez `00_setup`:

Parametr	Wartość
`catalog`	`retailhub_<nazwa>`
`source_path`	`/Volumes/retailhub_<nazwa>/default/datasets`



Dodaj taki sam wstrzymany harmonogram jak w jobie z M05 i wybierz **Run now**. Każdy task pokazuje swój wynik w zakładce **Output**, na przykład zapisaną tabelę i liczbę wierszy. Jeśli task się nie powiedzie, popraw notebook i użyj **Repair run**: ponownie uruchomią się tylko task, który zawiódł, i taski po nim.

Sprawdzanie wyników w SQL Editorze

Otwórz **SQL Editor**, wybierz **Serverless Starter Warehouse** i uruchom:

```
SELECT * FROM retailhub_<nazwa>.gold.lab_daily_orders ORDER BY order_date;

DESCRIBE HISTORY retailhub_<nazwa>.gold.lab_daily_orders;
```

Jak zmieścić się w limitach

- Uruchamiaj jeden job naraz: Free Edition pozwala na pięć tasków jednocześnie.
- Zostawiaj harmonogramy w stanie **Paused**. Codzienny harmonogram zużywa limit także po zakończeniu szkolenia.

CZĘŚĆ 6

Różnice względem szkolenia z trenerem

Szkolenie z trenerem odbywa się w Azure Databricks na współdzielonym klasycznym klastrze. Notebooki rozpoznają Serverless i obsługują znane różnice; w komórkach, których to dotyczy, pojawia się wyjaśnienie zamiast błędu.

Moduł	Komórka	We Free Edition
M01	Demo kosztów (trener)	Odczytuje `system.billing.usage`; przy błędzie uprawnień albo braku tabeli pomiń je
M01	BONUS Cluster Info and Monitoring	Runtime `client.x.y`; Photon jest zawsze włączony
M02	BONUS Ustawienia AQE	Da się odczytać tylko część ustawień Spark; dla pozostałych pokazywane są wartości domyślne
M02	BONUS Cache and Persist	Public Preview na Serverless (środowisko w wersji 5 lub nowszej): wynik liczony od razu jako snapshot sesji; w przeciwnym razie komórka wypisze komunikat
M02	BONUS Spark UI	Brak Spark UI; użyj profilu zapytania komórki
M03	BONUS Global Temporary Views	Nieobsługiwane na Serverless; pomijane
M04	BONUS VACUUM	Demo skraca retencję swojej tabeli demonstracyjnej, a potem ją przywraca
M04	BONUS External Tables	Brak external location; komórka to zgłasza
M06	GRANT / REVOKE	Tryb jednoosobowy: uprawnienia dla `account users` zamiast `retailhub_analysts`
M05, projekt końcowy	Compute jobów	Serverless zamiast klastra

Odczyt plików z volume'ów, Delta Lake, MERGE, time travel, RESTORE, joby z parametrami i task values oraz SQL Editor powinny działać tak samo. Status testów: w toku, dopóki ta edycja szkolenia nie zostanie uruchomiona we Free Edition od początku do końca.

CZĘŚĆ 7

Rozwiązywanie problemów

Objaw	Przyczyna	Rozwiązanie
`Catalog 'retailhub_<nazwa>' not found`	Nie uruchomiono `00_pre_config` albo używasz innego konta	Uruchom `00_pre_config` na koncie, którego używasz na szkoleniu
`%run ../setup/00_setup` kończy się błędem *notebook not found*	Notebook skopiowano poza Git folder	Otwórz notebook z folderu `databricks-fundamental`
Volume `datasets` jest pusty	Kopiowanie danych w `00_pre_config` nie zakończyło się	Uruchom `00_pre_config` ponownie
`CONFIG_NOT_AVAILABLE` albo `NOT_SUPPORTED_WITH_SERVERLESS`	Funkcja lub ustawienie nieobsługiwane na Serverless, często w starszej kopii repozytorium	Pobierz najnowszą wersję (część 3, *Pobieranie aktualizacji*); jeśli to komórka BONUS, pomiń ją
Komórka CHECK nie przechodzi	Zadanie jest niedokończone albo zmienna ma inną nazwę	Przeczytaj komunikat asercji, popraw komórkę zadania, uruchom obie komórki ponownie
`TABLE_OR_VIEW_NOT_FOUND` dla `lab_orders` w Silver lub Gold	Poprzednia warstwa nie została zbudowana	Najpierw uruchom task Bronze, potem Silver
Task `transform` albo `report` w jobie z M05 kończy się błędem	Taski mają inne nazwy niż `validate` i `transform`	Zmień nazwy tasków (część 5), potem **Repair run**
Compute jest niedostępny do końca dnia	Przekroczono dzienny limit zużycia	Poczekaj do następnego dnia; dane i notebooki zostają
Pull zgłasza konflikt	Notebook zmieniony lokalnie zmienił się też w repozytorium	Sklonuj swój notebook gdzie indziej, potem **Reset** i **Pull**

CZĘŚĆ 8

Sprzątanie

Po zakończeniu szkolenia usuń to, co zostało utworzone. Aby później zacząć od nowa, wystarczy ponownie uruchomić `00_pre_config` i `00_setup`.

1. W **Jobs & Pipelines** usuń job z M05 i job projektu końcowego albo przynajmniej zostaw ich harmonogramy wstrzymane.
2. Usuń Genie Agenta i dashboard z M07, jeśli zostały utworzone.
3. Usuń katalog szkoleniowy w notebooku albo w SQL Editorze:

```
DROP CATALOG IF EXISTS retailhub_<nazwa> CASCADE;
```

4. Opcjonalnie usuń Git folder `databricks-fundamental`.